When AI Agents Talk!

Inside the Agent-to-Agent (A2A) Protocol

Mar'2026

Speaker

Ashadeepa Debnath

Senior Software Architect
NiCE, Pune, India

Technologist, Passionate, Creative

Speaker

Manasi Purkar

Specialist Product Manager NiCE,
Pune, India

- Professional with nearly 2 decades of experience
- Developer, Architect, Product Manager
- Creative Keeda - Voracious Reader, Love to paint
- Mother to an 11-year-old who keeps me extremely active

Your 45-Minute Journey

Agenda

01

Agents just about everywhere !

02

Agents –
Be like Joy ☺

03

Why Agents need to talk ?

- MCP
- A2A – Language between Agents
- A2UI

04

When to use which ?

01

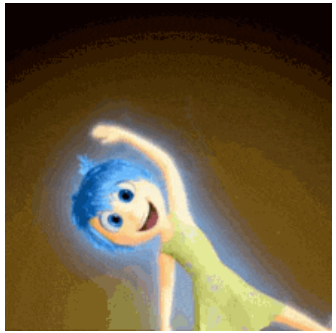
Agents Everywhere

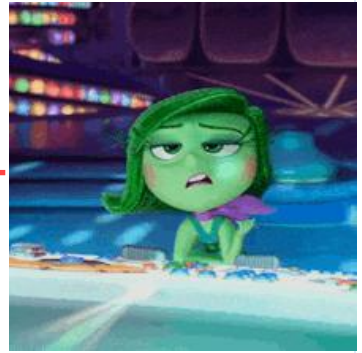
Meet Your AI Workforce 🎬

The Inside Out cast -as your enterprise agents. Every org already has all 6 — they just don't know it yet

The Problem: Agents Are Brilliant Loners 🏝️

Each agent is great at their job. But they can't talk to each other — or use tools outside their bubble





❌ Can’t share how to be happy

❌ Duplicate data calls

❌ No common task format

💡 **We need protocols. Standards. A way for agents to communicate, use tools**

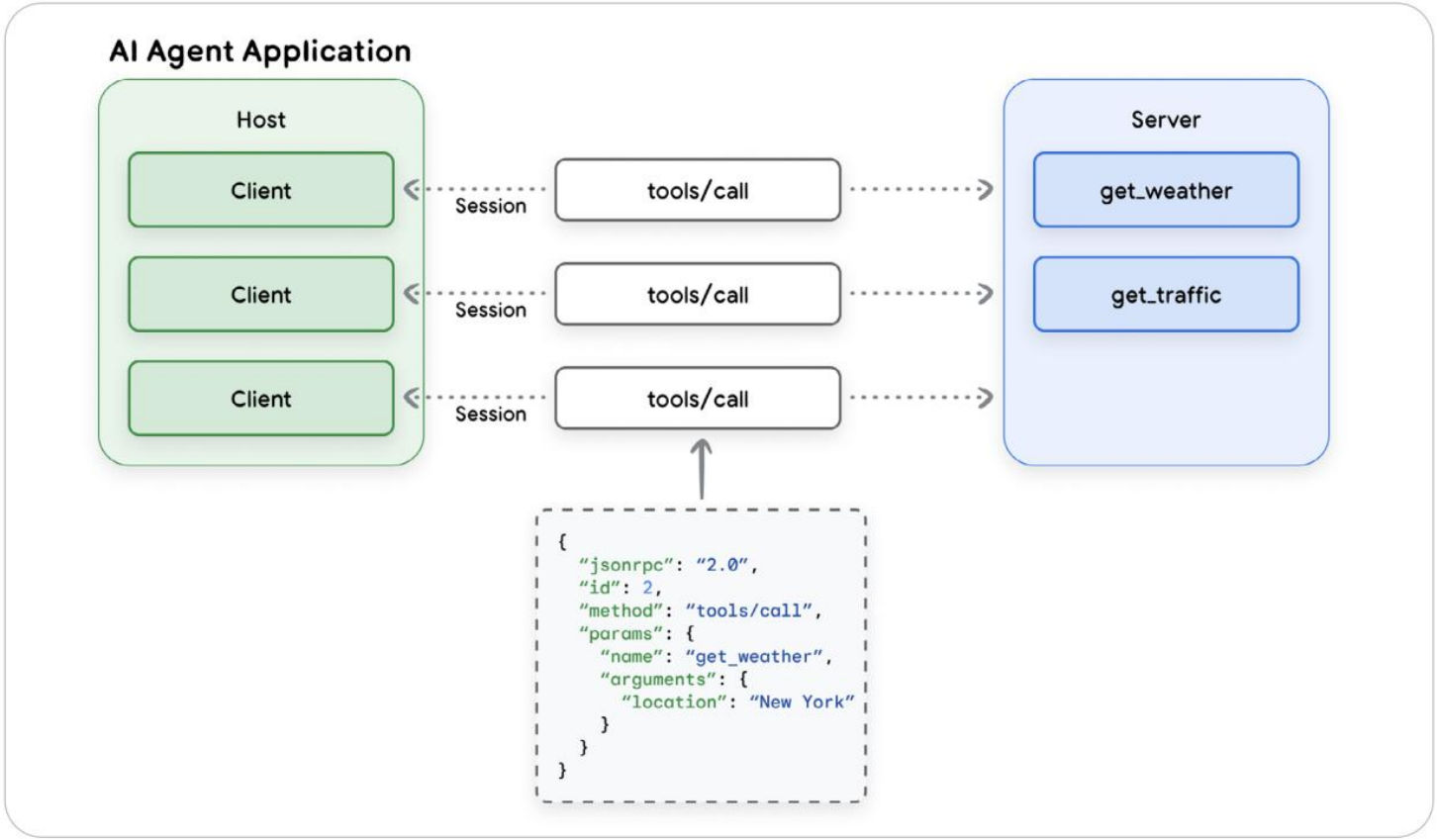
Your mission if you choose to accept it !

Keep emotions in Check !

02

MCP Tool Belt

Core Architectural Components



Remember - Hosts, Clients, and Servers

MCP's Three Primitives

Everything an agent needs to interact with the world — exposed via one protocol

RESOURCES

Data AI can READ

- → Tables & DB schemas
- → Documents & PDFs
- → File system contents
- → API responses
- → Memory stores

PROMPTS

Interaction templates

- → Query patterns
- → Best practice guides
- → Few-shot examples
- → Reusable instructions
- → Domain knowledge

TOOLS

Functions AI can CALL

- → run_query(sql)
- → send_email(to, body)
- → create_ticket(title)
- → insert_record(data)
- → read_file(path)

How MCP Works: The Request Flow

LLM / Host

(Claude, GPT, Gemini)

- Lists available tools
- Selects which to call
- Parses tool results
- Continues reasoning

MCP Protocol

① list_tools / list_resources

② call_tool(name, args)

③ tool_result { content }

MCP Server

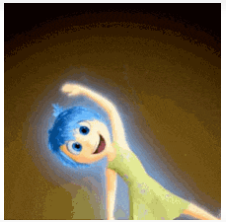
(Your tool provider)

- Exposes tools/resources
- Validates inputs
- Executes operations
- Returns structured data

Transport: stdio (local) | SSE (remote) | HTTP Streaming

MCP in the Enterprise: Vertical Integration

Each specialized agent owns its domain, connected to its tools via MCP. No horizontal chaos



MCP ✓

Tools

- Ticketing
- CRM
- Email

MCP ✓

Tools

- Docs
- Search
- Wiki

MCP ✓

Tools

- Payments
- Invoices
- Stripe

MCP ✓

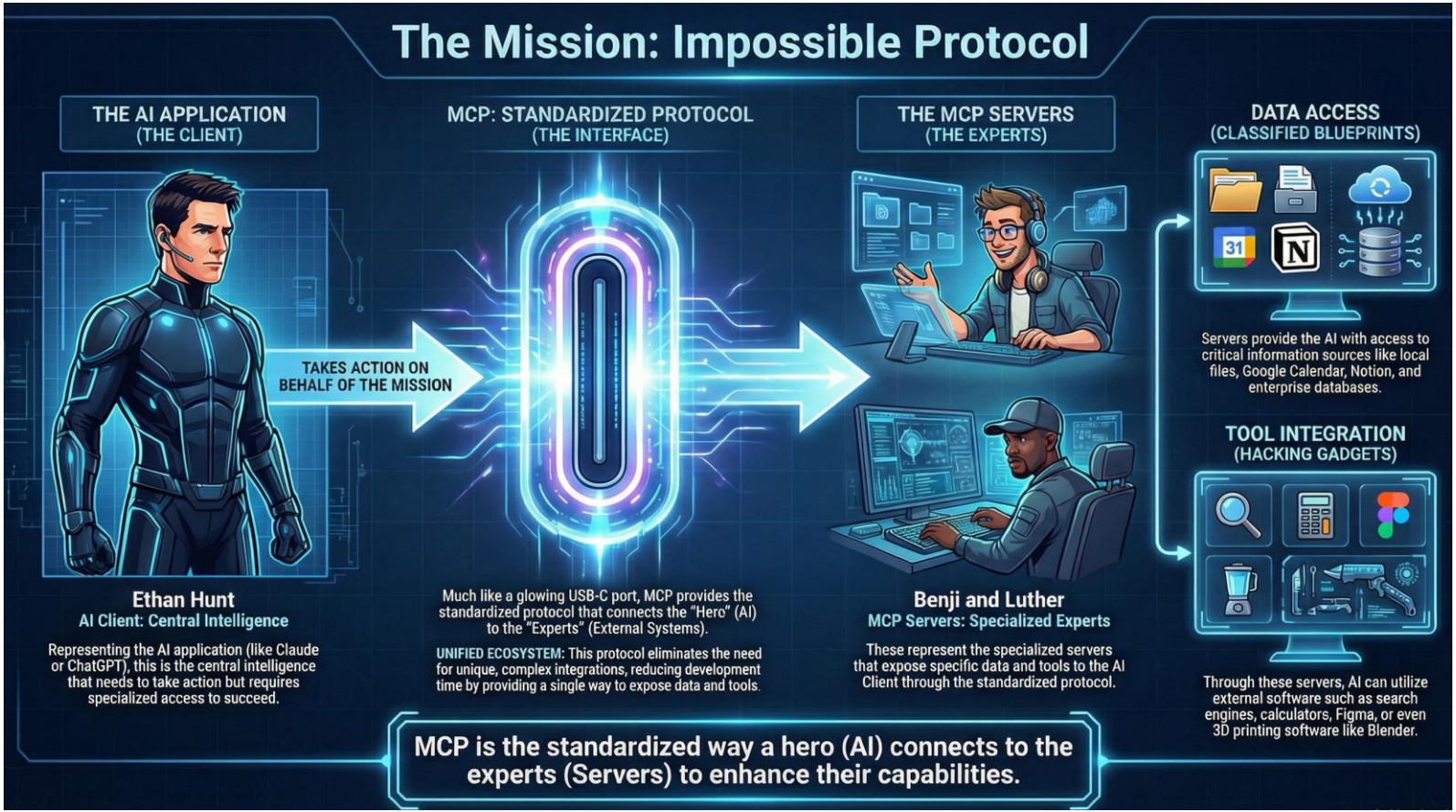
Tools

- GitHub
- Logs
- Infra

Each agent is specialized. Each connects to its tools via MCP. No agent needs to know how the others work

NiCE

Executes your agent's mission by using right tools



Tell me the mission and I know how to accomplish it ☺

03

A2A – Agents Talking to Agents

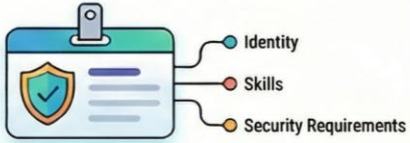
A2A Protocol — The Standard for AI Agent Collaboration

An open standard for secure AI agent collaboration on complex tasks without sharing internal states, using standard formats like JSON-RPC, gPRC, and REST

Core Architecture & Discovery

The Agent Card Manifest

A self-describing JSON document defining an agent's identity, skills, and security requirements.

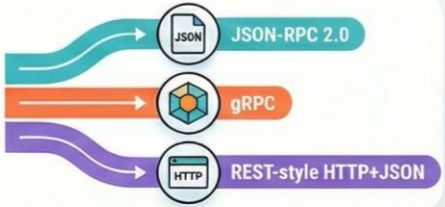


Opaque Execution Principle

Agents collaborate via declared capabilities without accessing each other's internal memory or tools.

3 Primary Transport Options

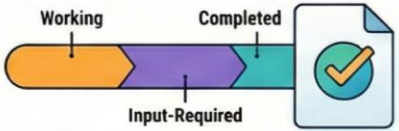
Full interoperability across JSON-RPC 2.0, gRPC, and REST-style HTTP+JSON protocols.



The Task & Interaction Lifecycle

Stateful Task Management

Tasks progress through defined states like "working," "input-required," and "completed."

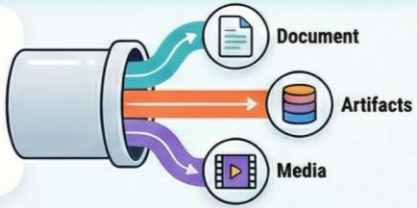


Flexible Interaction Modes

Interaction Mode	Synchronous	Streaming	Asynchronous
Delivery Method	HTTP POST	Server-Sent Events (SSE)	Webhock Push
Best Use Case	Quick queries or simple data retrieval.	Real-time updates for long-running generation	Disconnected or extremely long tasks.

Artifact-Based Results

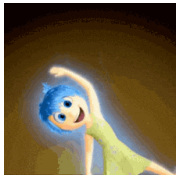
Agents return tangible outputs (documents, data, or media) as "Artifacts" within the task.



The Agent Card — Your Agent's Business Card

Published at /.well-known/agent.json — discoverable by any agent on the network

{ agent.json }



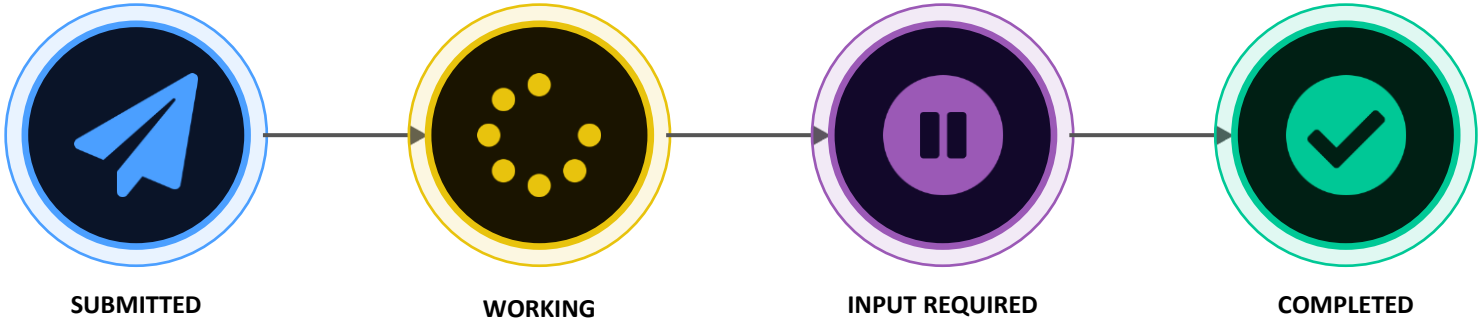
"name":	”Joy"
"description":	”Loves to make you Happy !
"url":	"https://cs.api.acme.com"
"version":	"2.1.0"
"skills":	[{id, description, tags}]
"authentication":	{ schemes: ["oauth2"] }
"defaultInputModes":	["text","json"]

- Unique identity in the network
- Natural language capability description
- Where to send tasks
- Capability version for compatibility
- What tasks this agent can accept
- How to authenticate with this agent
- Accepted input formats

💡 **Any agent that finds this card knows exactly how to delegate work — no custom integration needed.**

A2A Task Lifecycle — Request to Result

Tasks are async, stateful, and can run for minutes — or weeks. Designed for real enterprise workflows



Minutes to weeks

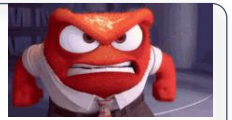
Check progress anytime

Resume after interruption

A2A: When Joy Needs to Call Anger

MCP lets agents use tools. A2A lets agents delegate to OTHER agents — as equals, with trus.

🎬 The Story



1. User asks: 'Why was I charged twice, and my support ticket is still open?'
2. Orchestrator detects billing issue + support issue → needs 2 agents
3. Orchestrator sends task to Billing Agent via A2A protocol
4. Simultaneously delegates to Customer Service Agent via A2A
5. Both agents work, return structured responses
6. Orchestrator aggregates, returns one coherent answer to user

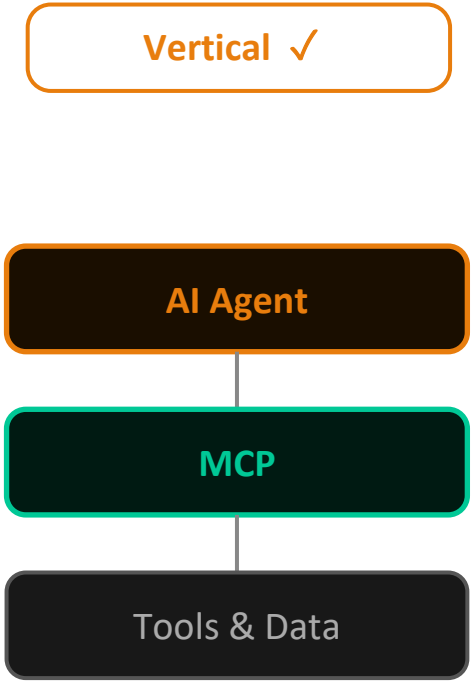
🎯 Why A2A Changes Everything

- 🔍 Agents find each other via Agent Cards at /.well-known/agent.json
- 📋 Standard format: ID, input, context, history
- ⏳ Tasks run minutes to weeks — check back anytime
- 🔒 OAuth2/mTLS — each agent is a secure, identifiable party
- 🌐 You don't need to know how an agent works, just its card
- 📜 Tasks can include text, images, files, structured data

A2A ❤ MCP

Feature	Agent-to-Agent (A2A)	Model Context Protocol
Relationship	Peer-to-Peer (Horizontal)	Client-to-Server (Vertical)
Focus	Collaboration and Delegation	Data Access and Tooling
Analogy	Two coworkers discussing a project	A worker reaching into a toolbox
Standard	Often relies on frameworks like LangGraph, AutoGen, or emerging IEEE standars.	Open-standard (JSON-RPC) for universal data connectors.

Vertical vs Horizontal Integration



Agent → Tools

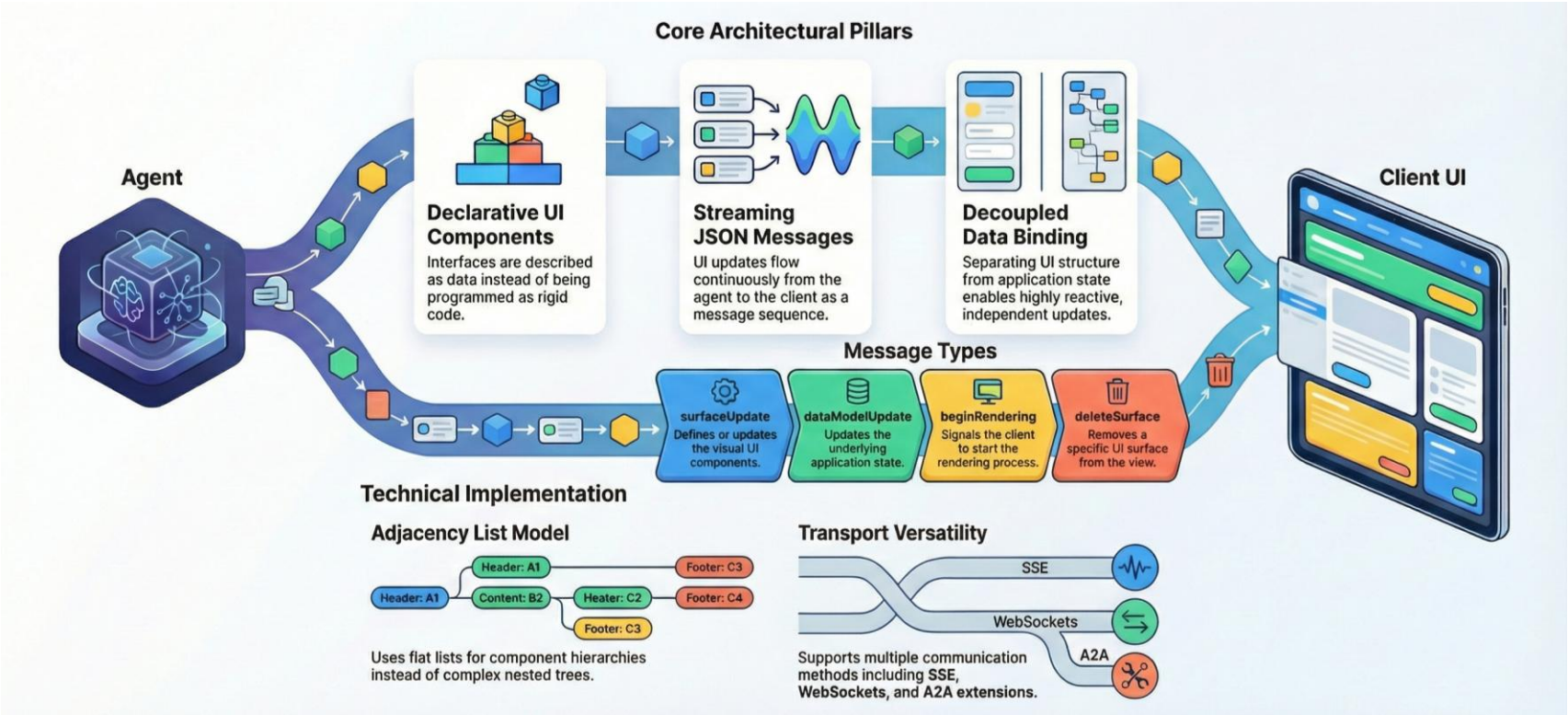
Agent ↔ Agent

04

A2UI & AGUI

A2UI – The Architecture of Agent-Driven Interfaces

A framework for reactive, agent-driven interfaces, shifting from statis coding to dynamic, data-driven models streaming JSON messages



A2UI vs AG-UI — Different but Complementary

Confusingly named. Very different roles. Both are needed in a production agentic app

A2UI

By Google + CopilotKit

- Declarative UI spec from agent responses
- Agent returns JSON component schema
- Frontend resolves to native components
- Platform-agnostic: web, mobile, desktop
- https://github.com/google/A2UI

AG-UI

By CopilotKit

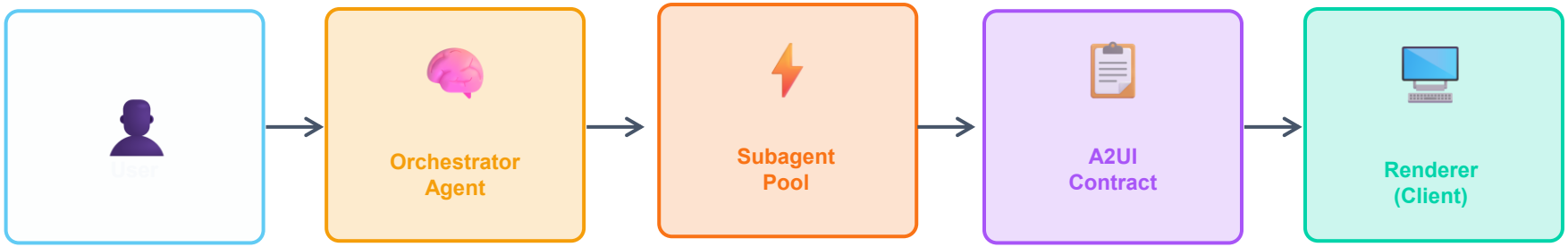
- Connects frontend apps to agentic backends
- Event-stream protocol (JSONL over HTTP)
- Handles streaming, tool calls, state updates
- Works with LLMs LangGraph, CrewAI, any agent
- https://github.com/ag-ui-protocol/ag-ui

05

The Full Agentic Stack

SubAgents→ A2UI: Who Does What

Agents don't render UI. They produce A2UI JSON. The client decides how to render it



🔑 Golden Rule

Subagents produce A2UI JSON.
They NEVER directly render UI.
All responses share the same schema.

Subagent responsibilities:

restaurant_finder	→ list / cards
contact_lookup	→ form / profile
rizzcharts	→ charts / tables
booking_agent	→ calendar widget

A2UI Renderer resolves:

✓ Component type → native widget

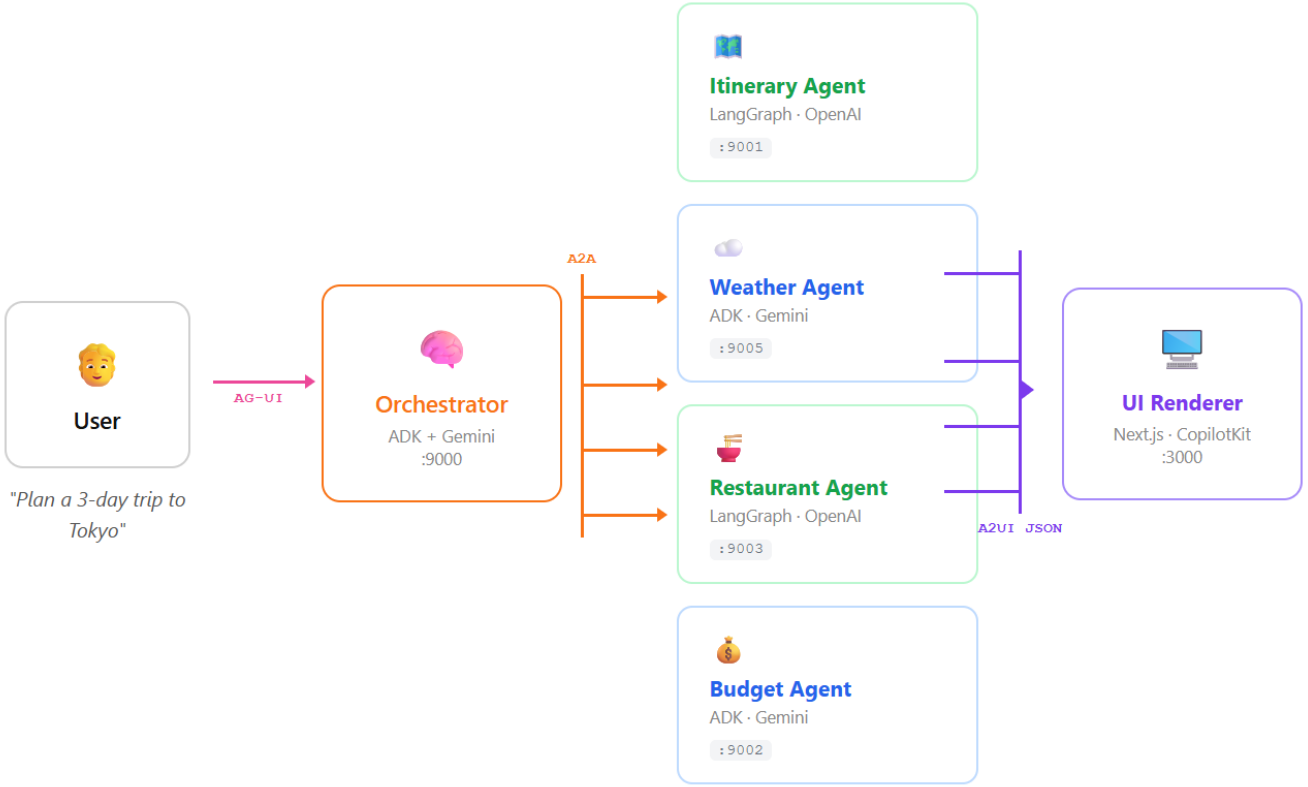
✓ Data bindings → live props

✓ Actions → event handlers

✓ Platform → Angular/React/Flutter/etc.

The Full Stack: AG-UI + A2A + A2UI in Action

A real travel planning demo — one user request, four specialized agents working in concert



https://github.com/Ashadeepa/a2a-demo-travel-app

06

Conclusion

Decision Guide: Which Protocol Do I Need?

Use this as a quick reference. Most production systems use all three.

My agent needs to call a DB / API / filesystem

MCP

Tools and data access = MCP primitives

My agent needs to ask another agent to do a task

A2A

Agent-to-agent delegation = A2A tasks

My agent should return clickable UI widgets

A2UI

Declarative UI from agent responses

I'm building a frontend that talks to an agent backend

AG-UI

Frontend ↔ agent event streaming

I need agents to discover each other dynamically

A2A

Agent Cards at /.well-known/agent.json

I want any LLM to use my custom tool

MCP

MCP server = universal tool interface

I need a long-running async multi-step workflow

A2A

A2A task lifecycle supports weeks-long tasks

My app needs to work on web, mobile, and desktop

A2UI

A2UI renderer adapts to any platform

Key Takeaways

1. Agents already exist — you're just not calling them that yet. Joy, Anger, Fear — they're all in your org.

2. MCP is the tool belt. It gives any agent a standard way to use tools, read data, and follow templates.

3. A2A is the language between agents. Agent Cards + Task Lifecycle = async, trusted, scalable delegation.

4. A2UI separates agent intent from rendering. Your agent returns structure; the UI decides how to show it.

5. All three compose together. MCP (tools) + A2A (agents) + A2UI (interface) = the complete agentic stack.

6. This is converging fast. Start with MCP (lowest friction), plan for A2A, design A2UI-ready agent outputs.

Accomplished ! Thank you

Agreement , Negotiation and more !

https://www.linkedin.com/in/ashadeepa-debnath/

https://www.linkedin.com/in/manasi-purkar/